

Recherche de documents « Document Retrieval »

Séance 3

Tokenisation, filtrage et normalisation lexicale

I. Introduction :

Au cours des deux premières séances, vous avez successivement :

- structuré et exploré un corpus bilingue organisé en sous-dossiers (Séance 1) ;
- développé une librairie riche de fonctions de nettoyage et de normalisation (Séance 2), afin d'obtenir des textes homogènes et cohérents pour un traitement linguistique ultérieur. Cette librairie, bien que robuste, n'est pas exhaustive. Certains éléments complexes n'ont pas encore été traités, tels que
 - les expressions temporelles (dates, heures),
 - les mots composés (« machine learning »)
 - et les verbes à particules (*phrasal verbs* comme « take off » ou « put up with »).

Ces phénomènes linguistiques nécessitent une analyse plus fine du lexique et de la structure des phrases.

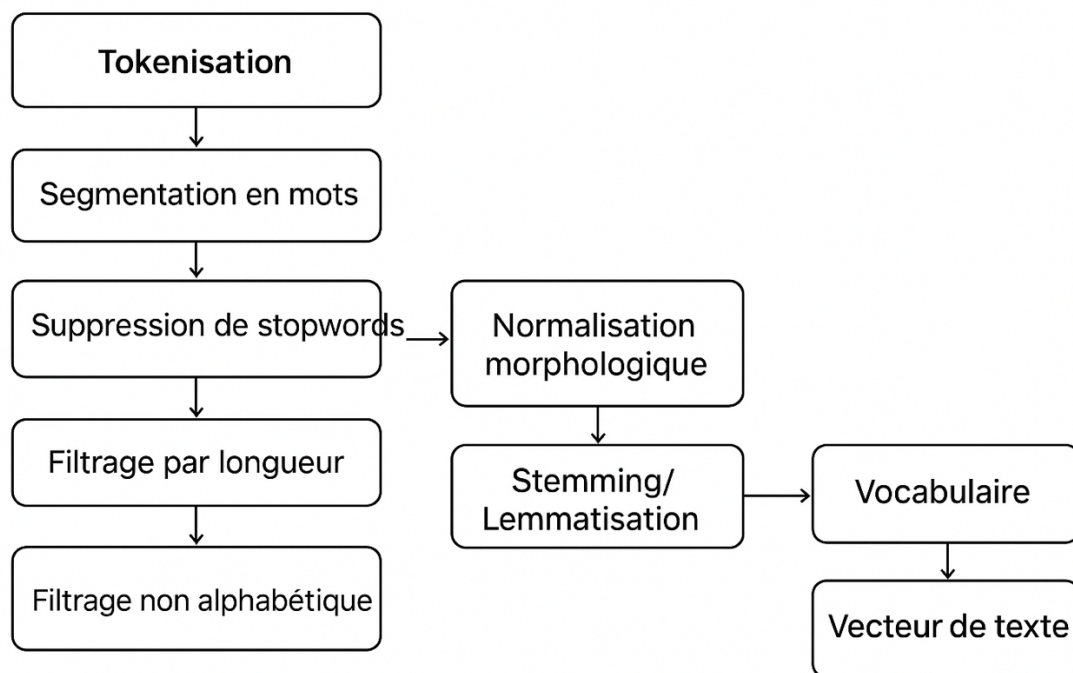
Par exemple, si on prend le texte « Le 15 août 2026, Dr. Martin ira à St. Martin. Il aime coder en Python! » ; La date, les abréviations (Dr., St.) et la ponctuation doivent être correctement gérées pour que la tokenisation en phrases et en mots soit précise.

L'objectif de cette troisième séance est de passer du texte nettoyé aux unités lexicales exploitables pour l'indexation et la recherche documentaire. Vous allez transformer les textes en représentations lexicales normalisées en implémentant vos propres fonctions Python pour :

- segmenter le texte en unités linguistiques (phrases, mots et n-grammes) ;
- calculer des statistiques lexicales de base (fréquences, tailles moyennes, richesse lexicale, hapax, etc.) ;
- filtrer et normaliser les tokens selon différents critères linguistiques ;
- appliquer manuellement des opérations de stemming et de lemmatisation simplifiée, sans utilisation de bibliothèques NLP ;
- construire un vocabulaire à partir de l'ensemble du corpus.
- évaluer l'impact des transformations successives sur la diversité lexicale et la structure du lexique.

Cette séance constitue le pont entre le prétraitement linguistique et la représentation vectorielle, qui sera abordée lors de la prochaine étape du projet.

Afin d'offrir une vision claire des différentes étapes de prétraitement étudiées dans cette séance, le schéma suivant présente l'enchaînement complet des opérations appliquées au corpus.



Vue d'ensemble du pipeline de prétraitement lexical

II. Plan de la séance

1. Tokenisation

La tokenisation consiste à découper un texte en unités linguistiques élémentaires appelées tokens. Elle représente la première étape d'analyse linguistique après le nettoyage effectué lors de la séance précédente. Une tokenisation bien conçue est essentielle car elle conditionne la qualité de toutes les étapes ultérieures (statistiques lexicales, indexation, recherche documentaire, etc.).

a. Tokenisation en phrases

La tokenisation en phrases vise à segmenter le texte selon la ponctuation, tout en évitant les découpages erronés dus aux abréviations « Dr., M., etc. », aux sigles « O.R.L, P.D.G », aux nombres décimaux « 3.14 », aux dates « 12.05.2025 » ou encore aux heures « 12:05:25 ».

La séance 2 n'a pas couvert l'identification et la normalisation des décimaux, dates, heures et sigles. Vous pouvez enrichir votre bibliothèque de nettoyage pour rendre la tokenisation plus robuste. Cette extension permettra d'obtenir une tokenisation en phrases plus précises et de réduire les erreurs lors des étapes ultérieures.

La segmentation en phrases permet d'obtenir une structure plus fine, facilitant ensuite la segmentation en mots et l'analyse du contexte.

- Écrire une fonction **segmenter_phrases** (**texte**, **abreviations**=[], **option** = { **"gerer_decimaux"**: False, **"gerer_dates"**: False, **"gerer_sigles"**: False }) qui segmente un texte en phrases en respectant la ponctuation (?!) et en évitant les erreurs dues aux abréviations, décimaux, dates ou sigles. Le découpage se fait à partir des signes de ponctuation (. ? !) suivis d'un espace ou d'un retour à la ligne. Cette fonction retourne une liste de phrases (chaînes).
- Vous pouvez utiliser des fonctions auxiliaires telles que :
 - **est_une_abreviation(token, abreviations)**,
 - **est_un_sigle(token, sigles)**,
 - **est_une_date(token)**,
 - **est_un_decimal(token)**

Exemple :

```

texte = "Dr. Martin habite à Reims. Il aime le chocolat!"
abreviations = ["Dr.", "M."]
segmenter_phrases(texte, abreviations)

Retour → ["Dr. Martin habite à Reims.", "Il aime le chocolat!"]

```

b. Tokenisation en mots

Après la segmentation en phrases, l'étape suivante consiste à découper chaque phrase en unités lexicales élémentaires, appelées *mots* ou *tokens*. Cette opération est essentielle pour construire des représentations lexicales exploitables : statistiques lexicales, vocabulaire, index, ou encore calculs de similarité.

La tokenisation en mots doit prendre en compte certaines particularités linguistiques. Certaines d'entre elles ont déjà été partiellement traitées lors de la séance 2 à travers des fonctions de nettoyage, qu'il est désormais important d'intégrer dans le pipeline avant la segmentation :

- **Les contractions (élisions et apostrophes) (*l'article*, *j'aime*, *don't*) :**

Déjà traitées par la fonction *expand_contractions(texte, langue)* développée en séance 2. Avant toute tokenisation, le texte doit passer par cette fonction afin de garantir une segmentation correcte.

- **Les ponctuations attachées aux mots (*bonjour!*, «*mot*», *etc.*) :**

Déjà traitées par la fonction *espacer_ponctuation(texte)* développée en séance 2.

- **La gestion des espaces multiples ou spéciaux :**

Les fonctions de la section « Normalisation des espaces et de la mise en forme » de la séance2 suppriment les espaces superflus, les tabulations et les retours à la ligne inutiles, assurant une segmentation propre et régulière.

Certains mots contiennent un tiret (*porte-monnaie*, *chef-d'œuvre*) : ces mots doivent idéalement être considérés comme un seul token, sauf si le contexte impose le contraire. D'autres expressions multi-mots comme *machine learning* ou *New York* peuvent être traitées comme des unités lexicales spécifiques (par exemple, via une liste d'expressions protégées ou lors d'une phase ultérieure d'analyse).

Ces étapes préliminaires garantissent un texte normalisé, permettant ensuite une segmentation en mots propre et cohérente.

- Écrire une fonction **segmenter_mots(phrase, balise = False)** qui découpe une phrase en une liste de tokens (mots). On suppose que le texte a été préalablement nettoyé et que les contractions ont été développées. L'option balise indique si les balises doivent être conservées comme des tokens. Elle retourne une liste de chaînes (mots/tokens)
- Écrire une fonction **tokeniser_document(texte, abbreviations=[], option = { "gerer_decimaux": False, "gerer_dates": False, "gerer_sigles": False }, balise = False)** qui segmente un document en phrases, ensuite segmente chaque phrase en mots. Elle retourne une liste de listes de chaînes (mots/tokens).
- Écrire une fonction **tokeniser_corpus(corpus, abbreviations=[], option = { "gerer_decimaux": False, "gerer_dates": False, "gerer_sigles": False }, balise = False)** qui segmente un corpus de documents. Le corpus est représenté par un dictionnaire où les clés sont les identifiants des documents et les valeurs sont les textes des documents (corpus : dictionnaire { id_doc : texte_doc }). La fonction retourne un dictionnaire de sorte que les clés restent les identifiants des documents et les valeurs sont les listes de listes de chaînes (mots/tokens).
- Écrire une fonction **aplatir_tokens(document_tokens)** qui aplatit une liste de listes de tokens en une seule liste.
- Écrire une fonction **tokens_hapax(document)** pour identifier les *hapax*, c'est-à-dire les tokens apparaissant une seule fois dans le texte.

c. Tokenisation en n-grammes

La tokenisation en n-grammes permet de prendre en compte la proximité et l'ordre des mots dans un texte.

Elle est utile pour :

- capturer des séquences lexicales fréquentes,
- enrichir le vocabulaire,
- construire des représentations vectorielles plus contextuelles,
- améliorer la recherche documentaire et les modèles statistiques.

Définitions :

- Unigrammes : mots isolés (tokens simples)
- Bigrammes : paires de mots successifs
- Trigrammes : séquences de trois mots consécutifs
- N-grammes : séquences de N mots consécutifs (généralisation des cas précédents)

Avant de générer des *n*-grammes, il est essentiel que le texte ait été nettoyé et normalisé comme en séance 2 :

- expansion des contractions,
 - espacement correct de la ponctuation,
 - normalisation des espaces et de la casse,
 - gestion des balises de protection <NUM>, <EMAIL>, <URL>, etc.
- Écrire une fonction générique **generer_ngrammes(donnees, n, niveau = 'phrase', par_phrase = True)** qui permet de générer les *n*-grammes à différents niveaux d'analyse selon la structure du jeu de données. La structure des données reçues dépend du niveau d'analyse choisi :
 - **niveau = 'phrase'** : une liste de tokens.
 - **niveau = 'document'** : une liste de listes de tokens.
 - **niveau = 'corpus'** : un dictionnaires {id_doc : liste de listes de tokens}

Elle reçoit aussi un entier $n \geq 1$ qui correspond à la taille de la séquence, et l'option **par_phrase : booléen** indiquant s'il faut respecter les limites syntaxiques entre phrases ou les ignorer.

La fonction traitera les données selon le niveau et l'option, c'est-à-dire :

- **niveau = "phrase"** : les *n*-grammes sont générés directement à partir de la liste de tokens de la phrase.
- **niveau = "document"** :
 - **par_phrase = True** : les *n*-grammes sont générés indépendamment pour chaque phrase, puis agrégés.
 - **par_phrase = False** : les tokens de toutes les phrases du document sont concaténés avant la génération, ce qui permet de capturer des enchaînements traversant les limites de phrases.
- **niveau = "corpus"** :
 - **par_phrase = True** → les *n*-grammes sont produits séparément pour chaque document (et éventuellement par phrase).
 - **par_phrase = False** → les tokens de chaque document peuvent être concaténés, ou l'ensemble du corpus aplati pour obtenir une analyse globale sans tenir compte des frontières entre documents.

La fonction retourne soit une liste de tuples représentant les *n*-grammes (pour un document ou une phrase), soit un dictionnaire {id_doc: liste de *n*-grammes} lorsqu'elle est appliquée à un corpus.

Exemple sur une phrase:

```
tokens = ["Le", "chat", "dort", "paisiblement"]

# Unigrammes (N=1)

generer_ngrammes(tokens, niveau= 'phrase', 1, par_phrase=True)

Retour → [("Le",), ("chat",), ("dort",), ("paisiblement",)]

# Bigrammes (N=2)

generer_ngrammes(tokens, niveau= 'phrase', 2, par_phrase=True)

Retour → [("Le","chat"), ("chat","dort"), ("dort","paisiblement")]

# Trigrammes (N=3)

generer_ngrammes(tokens, niveau= 'phrase', 3, par_phrase=True)

Retour → [("Le","chat","dort"), ("chat","dort","paisiblement")]

generer_ngrammes(tokens, niveau= 'phrase', 4, par_phrase=True)

Retour → [("Le","chat","dort","paisiblement")]
```

2. Statistiques sur le corpus avant filtrage

L'objectif de cette partie est d'explorer le corpus après la phase de tokenisation, afin d'obtenir une vision quantitative et qualitative de sa structure lexicale avant toute opération de filtrage (suppression de stopwords, élimination des tokens non pertinents, stemming, lemmatisation, etc.). Ces statistiques serviront de référence pour évaluer l'impact des transformations linguistiques ultérieures.

Les mesures obtenues permettront notamment de :

- quantifier la richesse lexicale du corpus,
- identifier le bruit (tokens non significatifs ou vides),
- observer la distribution des fréquences lexicales,
- et préparer la construction du vocabulaire actif après filtrage.

a. Comptage global

Une fois le corpus tokenisé (en phrases puis en mots), la première étape consiste à effectuer des comptages globaux permettant de mesurer la densité lexicale et la structure brute du corpus.

Ces mesures offrent une vue d'ensemble sur :

- le volume de données textuelles à traiter,
 - la proportion de tokens utiles par rapport au bruit,
 - et la présence éventuelle d'un déséquilibre lexical entre les documents.
-
- Écrire une fonction **compter_tokens_phrase(phrase)** qui calcule le nombre de tokens présents dans une phrase.
 - Écrire une fonction **compter_tokens_document(document)** qui compte le nombre total de tokens dans un document (toutes les phrases confondues). Le document est représenté sous forme de liste de listes de tokens.
 - Écrire une fonction **compter_tokens_corpus(corpus)** qui agrège les résultats pour l'ensemble du corpus et retourne le nombre total de tokens présents dans tous les documents. Le corpus est un dictionnaire de la forme `{id_doc : document_tokenisé}`.
 - Écrire une fonction **calculer_mots_vides_document(document, stopwords)** qui compte le nombre de mots vides (*stopwords*) dans un document à partir d'une liste prédéfinie. Cette mesure permet d'estimer la proportion de mots non informatifs.
 - Écrire une fonction **calculer_tokens_vides_document(document)** qui identifie et compte le nombre de tokens vides ou non alphabétiques (" ", "!", "123", etc.) dans un document.
 - Écrire une fonction **calculer_mots_vides_corpus(corpus, stopwords)** qui compte le nombre de mots vide (*stopwords*) dans un corpus.
 - Écrire une fonction **calculer_tokens_vides_corpus(corpus)** qui compte le nombre de tokens vides ou non alphabétiques dans un corpus.

Ces fonctions permettent de mesurer le bruit lexical du corpus avant toute normalisation. Vous pouvez stocker les résultats sous forme d'un dictionnaire :

```
stats_corpus = {  
    "nb_documents": len(corpus),  
    "nb_tokens_total": ...,  
    "nb_mots_vides": ...,  
    "nb_tokens_vides": ...,  
    "proportion_bruit": (nb_mots_vides + nb_tokens_vides) / nb_tokens_total  
}
```

b. Distribution des tokens

Au-delà du comptage global, il est important d'analyser la distribution des tokens afin de mieux comprendre la structure interne du corpus : fréquence des mots, longueur des documents, variation de taille des phrases et richesse lexicale.

i. Analyse des longueurs

- Écrire une fonction **distribution_longueur_documents(corpus)** qui calcule la longueur (en nombre tokens) de chaque document et retourne un dictionnaire `{id_doc : longueur_document}`.
- Écrire une fonction **distribution_longueur_phrases(corpus)** qui calcule la longueur moyenne des phrases (en nombre tokens) pour chaque document et retourne un dictionnaire `{id_doc : longueur_moyenne}`.
- Écrire une fonction **distribution_longueur_mots(corpus)** qui calcule la longueur moyenne des mots (en caractères) et retourne la distribution de ces longueurs sous forme d'un dictionnaire `{longueur : fréquence}`.

ii. Analyses des occurrences

- Écrire une fonction **distribution_occurrences_tokens(corpus)** qui compte le nombre d'occurrences de chaque token dans le corpus et retourne un dictionnaire `{token : nb_occ}`.
- Écrire une fonction **tokens_plus_frequents(corpus, n = 20)** qui extrait les n tokens les plus fréquents du corpus (hors ponctuation et tokens vides).

Ces fonctions fourniront les bases statistiques nécessaires à la construction du vocabulaire et au calcul ultérieur des indicateurs lexicaux (richesse, hapax, dispersion) qui seront abordés dans la section suivante.

Ces indicateurs permettent d'évaluer la richesse lexicale et la structure du vocabulaire et d'évaluer la redondance lexicale avant filtrage.

c. Moyennes et dispersion

Pour compléter l'analyse, il est utile de calculer des indicateurs de tendance centrale et de dispersion afin de caractériser la structure globale du corpus.

- Écrire une fonction **statistiques_corpus(corpus)** qui calcule les statistiques globales suivantes :
 - moyenne des longueurs de documents (en tokens),
 - écart-type des longueurs documentaires,
 - longueur maximale et minimale,
 - moyenne du nombre de phrases par document,
 - moyenne du nombre de tokens par phrase.

La fonction retourne un dictionnaire de la forme :

```
{  
    "longueur_moyenne_doc": ...,  
    "ecart_type_doc": ...,  
    "longueur_min_doc": ...,  
    "longueur_max_doc": ...,  
    "moyenne_phrases_par_doc": ...,  
    "moyenne_tokens_par_phrase": ...  
}
```

Ces indicateurs permettent d'obtenir une vision statistique synthétique du corpus avant normalisation.

d. Visualisation des statistiques

La visualisation constitue une étape clé pour interpréter les statistiques du corpus et identifier rapidement d'éventuelles anomalies, comme des documents trop courts ou trop longs, ou un vocabulaire excessivement redondant.

- Écrire une fonction `tableau_de_bord` (`stats_corpus`) qui génère un tableau de bord ou des représentations graphiques des statistiques calculées. Ce tableau peut inclure :
 - des histogrammes pour la distribution des longueurs de documents et des longueurs de mots,
 - des barplots pour les tokens les plus fréquents,
 - une courbe de fréquence des tokens,
 - des diagrammes circulaires (pie charts) pour comparer la proportion de tokens utiles versus tokens vides ou non pertinents.

Ces représentations graphiques permettent d'avoir une vue synthétique et intuitive du corpus avant toute opération de filtrage, facilitant la détection de déséquilibres lexicaux et de bruit textuel.

3. Construction du vocabulaire et diversité lexicale

a. Construction du vocabulaire

Après l'analyse de la distribution des tokens, l'étape suivante consiste à construire le vocabulaire brut, c'est-à-dire l'ensemble des formes lexicales uniques présentes dans le corpus avant tout filtrage.

Cette étape permet :

- de recenser la totalité des unités lexicales observées ;
 - de préparer les traitements de filtrage (stopwords, fréquence minimale, longueur, etc.) ;
 - et de constituer la base sur laquelle seront calculées les mesures de diversité lexicale.
- Écrire une fonction **construire_vocabulaire(corpus)** qui retourne la liste des tokens distincts (uniques) présents dans le corpus.
 - Écrire une fonction **vocabulaire_ngrammes(corpus, n)** qui retourne la liste des n-grammes uniques (bi-grammes, tri-grammes, etc.) pour enrichir l'analyse contextuelle.

Ces listes serviront ensuite à calculer les indicateurs lexicaux et à sélectionner les tokens pertinents après filtrage.

b. Diversité lexicale

La diversité lexicale renseigne sur la variété et la répartition du vocabulaire utilisé dans un corpus. Elle permet de comparer différents sous-corpus, langues ou genres textuels.

- Écrire une fonction **calculer_richesse_lexicale(vocabulaire)** qui mesure la diversité du vocabulaire :

$$\text{richesse} = \frac{\text{nombre de mots uniques}}{\text{nombre de tokens total}}$$

Une valeur élevée indique un corpus lexicalement riche et varié.

- Écrire une fonction **calculer_taux_hapax(vocabulaire)** qui calcule la proportion de mots apparaissant une seule fois :

$$\text{taux_hapax} = \frac{\text{nombre d'hapax}}{\text{nombre de mots uniques}}$$

Un taux d'hapax élevé indique une forte proportion de mots rares.

- Écrire une fonction **calculer_dispersion_lexicale(vocabulaire)** qui mesure la variabilité des occurrences des tokens dans le vocabulaire (écart-type des occurrences) reflétant la concentration ou l'équilibre du lexique.

Les résultats de ces fonctions peuvent être regroupés dans une structure synthétique : `stats_vocabulaire = { "richesse_lexicale": ..., "taux_hapax": ..., "dispersion_lexicale": ... }`

Ces indicateurs permettent ensuite de comparer la diversité lexicale :

- entre les quatre sous-corpus du corpus global,
- et entre les langues étudiées (FR / EN).

c. Conversion du vocabulaire en dictionnaire d'index

Une fois le vocabulaire final obtenu (avant et après filtrage), il est utile de le transformer en une structure de dictionnaire associant chaque mot à un identifiant unique.

- Écrire une fonction **construire_dictionnaire_vocabulaire(vocabulaire)** qui construit deux dictionnaires d'indexation du vocabulaire :
 - dictionnaire direct {mot : index}
 - dictionnaire inverse {index : mot}

Les indices sont typiquement consécutifs à partir de 0. Cette représentation facilitera l'encodage des documents en vecteurs pour les modèles de traitement automatique.

4. Filtrage lexical

Le filtrage vise à réduire le bruit lexical présent dans le vocabulaire brut afin de construire un ensemble de tokens plus informatif et représentatif du contenu réel du corpus. Contrairement au nettoyage typographique réalisé précédemment, cette étape agit sur le contenu lexical pour sélectionner les unités les plus pertinentes. Elle constitue une étape intermédiaire essentielle entre la construction du vocabulaire et la normalisation morphologique.

Différentes stratégies de filtrage peuvent être mises en œuvre pour mesurer leur impact sur la structure du lexique et, plus tard, sur les performances de recherche documentaire.

a. Suppression des mots vides (stopwords)

Les mots vides (*stopwords*) sont des termes très fréquents apportant peu ou pas d'information sémantique (ex. : *le, la, les, and, of, the*). Les supprimer permet de réduire le bruit lexical et de concentrer l'analyse sur les unités de contenu.

- Écrire une fonction **construire_liste_stopwords(langue)** qui construit une liste personnalisée de stopwords (mots non porteurs de sens « articles, prépositions, pronoms, etc. ») selon la langue (français et en anglais). Cette liste peut être enrichie selon le domaine étudié. Vous trouverez sur ce site différentes listes de mots vides : <https://www.ranks.nl/stopwords>
- Écrire une fonction **supprimer_mots_vides(tokens, liste_stopwords)** qui retire les mots vides « stopwords ».

b. Filtrage par longueur

Certains tokens très courts (ex. “a”, “de”, “la”, “en”) sont souvent peu informatifs, même s'ils ne figurent pas dans la liste des *stopwords*. Un filtrage par longueur minimale permet de se concentrer sur les unités lexicales significatives.

- Écrire une fonction **supprimer_tokens_courts(tokens, longueur_min=3)** qui retire les tokens dont la longueur est inférieure au seuil spécifié.

c. Filtrage des tokens non alphabétiques

Pour éliminer le bruit résiduel lié aux chiffres, symboles ou erreurs d'OCR, il est recommandé de supprimer les tokens contenant des caractères non alphabétiques.

- Écrire une fonction **supprimer_non_alphabetiques(tokens)** qui retire les tokens non alphabétiques.

d. Filtrage selon le nombre d'occurrences

Certains mots apparaissent de manière trop fréquente (bruit lexical) ou trop rare (hapax, erreurs, bruits d'OCR). Le filtrage par occurrences permet d'obtenir un vocabulaire plus équilibré et informatif.

- Écrire une fonction **filtrer_par_occurrence(vocabulaire, occ_min=2, occ_max=None)** qui conserve uniquement les tokens dont le nombre d'occurrences est compris entre les seuils définis, ajustables selon la taille du corpus ou le type de texte.

e. Pipeline de filtrage paramétrable

Pour éviter d'appliquer manuellement chaque fonction, vous allez concevoir une fonction unique qui orchestre automatiquement les étapes de filtrage selon une configuration donnée.

Cette fonction permettra de tester plusieurs combinaisons de filtres et de mesurer leur impact sur le vocabulaire final et la diversité lexicale.

- Écrire la fonction **pipeline_filtrage(tokens, config, langue="fr")** qui applique successivement les opérations activées dans un dictionnaire de configuration.
- Exemple de configuration :

```
config_experimentale = {
    "stopwords": True,
    "longueur_min": 3,
    "non_alphabetiques": True,
    "occ_min": 2,
    "occ_max": None
}
```

- Ce pipeline permet de tester plusieurs configurations et de mesurer leur impact sur :
 - la taille du vocabulaire,
 - la fréquence moyenne des mots,
 - la richesse lexicale.

L'objectif est d'identifier la configuration la plus équilibrée pour préparer la vectorisation et l'analyse lexicale.

5. Normalisation morphologique (Stemming et Lemmatisation)

Après le filtrage lexical, les tokens conservés peuvent encore présenter de nombreuses variations morphologiques : pluriels, conjugaisons, dérivations, etc. La normalisation morphologique regroupe ces formes différentes sous une même unité lexicale de base afin de réduire la dimension du vocabulaire et d'améliorer la cohérence sémantique. Elle comprend deux opérations principales :

- **Stemming**, basé sur des règles de troncature (règles simples sur les suffixes) ;
- **Lemmatisation**, qui repose sur des dictionnaires ou des règles linguistiques.

Ces étapes doivent être adaptées à la langue du texte (français ou anglais). Pour uniformiser les formes fléchies et dérivées tout en conservant un niveau de généralisation pertinent.

- Exemples

Formes originales	Après stemming	Après lemmatisation
étudier, étudiante, étudiés	étudi	étudier
running, runs, ran	run	run

Le stemming est plus rapide mais moins précis : il coupe les suffixes sans analyse linguistique. La lemmatisation est plus fiable mais nécessite des ressources (règles, dictionnaires). Dans ce projet, des versions simplifiées, manuelles et interprétables seront implémentées.

a. Stemming simplifié par règles

Un stemmer minimaliste peut être construit en supprimant des suffixes fréquents.

- Exemples de suffixes à traiter :
 - **Français** : *-ements, -ations, -ation, -ateur, -trice, -euse, -ement, -ment, -s, -es, -ions, -aient, etc.*
 - **Anglais** : *-ingly, -edly, -ing, -ed, -ly, -ness, -ers, -es, -s, -tion, -able, etc.*

Vous trouverez ces deux sites des listes de suffixes des deux langues :

- Français : https://en.wiktionary.org/wiki/Category%3AFrench_suffixes
- Anglais : <https://www.learnthat.org/pages/view/suffix.html>

- Écrire la fonction **appliquer_stemming(tokens, langue='fr')** qui crée un stemmer simple. Elle supprime le suffixe le plus long correspondant à la fin du token, sans raccourcir excessivement (exclure les mots de moins de 3 caractères). La fonction retourne la liste des stems. Vous pouvez représenter les règles sous forme d'un dictionnaire :

```
regles_suffixes = { "fr" : ["ements", "ations", "ation", "ement",
"ment", "euse", "trice", "s", "es"], "en" : ["ingly", "edly",
"ing", "ed", "ly", "ness", "ers", "es", "s"] }
```

b. Lemmatisation manuelle simplifiée

Le lemmatiseur ramène chaque mot à sa forme canonique (lemme). Dans cette version simplifiée, un dictionnaire minimal est utilisé.

Vous pouvez utiliser des lemmes de ces liens pour construire vos dictionnaires :

- <https://gitlab.irit.fr/ala/legal-term-extraction/-/blob/42898711228bc69b3ff14cb75a0eee60b97ddcaf/data/lefff-3.4-addition.csv>
- <https://www.kaggle.com/datasets/ianchute/english-lemmas>
- Écrire une fonction **appliquer_lemmatisation(tokens, langue="fr", dictionnaire_lemmes = None)** qui applique une lemmatisation manuelle avec un dictionnaire minimal selon la langue. Si un mot n'est pas dans le dictionnaire, il est retourné tel quel.
- Exemple de dictionnaires

```
lemmes_fr = {"étudiants": "étudiant", "mangeaient": "manger", "programmation":
"programme", "allées": "aller"}

lemmes_en = {"running": "run", "played": "play", "machines": "machine", "better":
"good"}
```

c. Intégration dans le pipeline morphologique

Pour tester l'impact de la normalisation morphologique, les étapes peuvent être intégrées dans un pipeline paramétrable.

- Écrire une fonction **pipeline_morphologique(tokens, config, langue = "fr", lemmes = None)** qui permet de configurer la normalisation morphologique

```
config_morpho = { "stemming" : True, "lemmatisation" : False }
```

- Créer plusieurs configurations morphologiques : stemming seul, lemmatisation seule, les deux combinés, aucun.

- Appliquer chaque pipeline sur le même corpus filtré.
- Comparer la taille du vocabulaire et le taux d'hapax.
- Identifier la stratégie offrant le meilleur compromis entre réduction du vocabulaire et préservation du sens lexical.

6. Statistiques après filtrage et normalisation

Une fois les opérations de filtrage lexical et de normalisation morphologique appliquées, il est essentiel d'évaluer leur impact sur la structure et la richesse du vocabulaire.

Cette étape remplit une double fonction :

- Vérifier la qualité du corpus préparé après transformation ;
- Quantifier les effets induits par chaque configuration du pipeline sur les caractéristiques lexicales.

a. Calcul des statistiques

À partir du corpus filtré et normalisé, vous allez réutiliser les fonctions développées dans les sections précédentes :

- section 2 (*Statistiques avant filtrage*),
- section 3 (*Construction du vocabulaire et diversité lexicale*).
- Écrire une fonction **calculer_statistiques_post_traitement(corpus_filtre, vocabulaire_filtre)** qui
 - compte le nombre total de tokens,
 - mesure la taille du vocabulaire final,
 - calcule la richesse lexicale,
 - estime le taux d'hapax,
 - calcule la proportion de mots vides résiduels (le cas échéant).

Elle retourne un dictionnaire de synthèse comparable à celui des statistiques initiales :

```
stats_post = {
    "nb_documents": len(corpus_filtre),
    "nb_tokens_total": ...,
    "taille_vocabulaire": ...,
    "richesse_lexicale": ...,
    "taux_hapax": ...,
    "proportion_bruit_residuel": ...
}
```

Ces indicateurs permettront de comparer le corpus avant et après traitement, afin de mesurer l'efficacité du pipeline lexical et d'identifier la configuration offrant le meilleur équilibre entre réduction du bruit et préservation de la diversité lexicale.

b. Visualisation des effets du prétraitement

Pour compléter l'analyse quantitative, il est recommandé de visualiser l'impact des différentes étapes de filtrage et de normalisation morphologique. Ces visualisations permettent de comparer de manière intuitive les distributions avant et après traitement, et d'évaluer l'efficacité des transformations appliquées.

Afin d'assurer la cohérence graphique avec les visualisations initiales, cette étape peut réutiliser et étendre la fonction `tableau_de_bord(stats_corpus)` définie à la section 2.

- Écrire une fonction **visualiser_statistiques_post_traitement(stats_initiales, stats_post)** qui s'appuie sur `tableau_de_bord()` pour afficher côte à côte les principales mesures (taille du vocabulaire, richesse lexicale, taux d'hapax, etc.). Produire, par exemple :
 - un barplot comparatif des tailles de vocabulaires avant et après traitement,
 - un graphique en barres groupées pour la richesse lexicale et le taux d'hapax,
 - un diagramme circulaire pour visualiser la réduction du bruit lexical.

Cette fonction ne duplique pas les visualisations de la section 2, mais les étend à une comparaison avant/après. Elle constitue ainsi une étape de validation visuelle du pipeline de prétraitement, avant la comparaison finale entre les différentes configurations expérimentales.

7. Comparaison des configurations de prétraitement

Après avoir calculé les statistiques post-traitement, il est essentiel de comparer plusieurs configurations complètes de prétraitement (combinant filtrage et normalisation morphologique) afin d'évaluer leur influence sur la structure lexicale du corpus.

Cette analyse constitue la base expérimentale de la séance 4, consacrée à l'étude de l'impact des prétraitements sur les performances de recherche documentaire (rappel, précision, similarité cosinus, etc.).

a. Génération de configurations multiples

Pour automatiser la comparaison, vous allez construire un pipeline configurable dans lequel chaque brique du prétraitement (filtrage, stemming, lemmatisation) peut être activée ou désactivée selon un dictionnaire de paramètres.

- Écrire une fonction **pipeline_pretraitement(corpus, config, langue="fr", stopwords=None, lemmes=None)** qui retourne un corpus traité selon une configuration donnée.

- Exemples de configurations à tester

Nom	Stopwords	Longueur min.	Non alphabétiques	Stemming	Lemmatisation
A	✗	✗	✗	✗	✗
B	✓	✓	✓	✗	✗
C	✓	✓	✓	✓	✗
D	✓	✓	✓	✗	✓
E	✓	✓	✓	✓	✓

Ces cinq configurations permettent d'évaluer progressivement l'impact de chaque composant de prétraitement.

b. Évaluation des effets

- Appliquer chaque configuration sur le même corpus propre obtenu lors de la séance 2, puis calculer pour chacune :
 - le nombre total de tokens,
 - la taille du vocabulaire,
 - la richesse lexicale,
 - le taux d'hapax,
 - et la proportion de bruit lexical.

Stocker les résultats dans une structure de synthèse :

```
resultats_configurations = {
  "A": {"nb_tokens": ..., "vocab": ..., "richesse": ..., "hapax": ..., etc.},
  "B": {...},
  "C": {...},
  "D": {...},
  "E": {...}
}
```

c. Analyse comparative

Interpréter les tendances observées entre les différentes configurations :

- Quels filtres réduisent le plus la taille du vocabulaire ?
- Quel traitement maintient une bonne richesse lexicale tout en réduisant le bruit ?
- Le stemming et la lemmatisation produisent-ils des effets similaires selon la langue (FR/EN) ?
- Quelle configuration semble offrir le meilleur compromis entre simplification morphologique et richesse lexicale ?

d. Visualisation comparative des configurations

Pour faciliter l'interprétation des résultats, il est recommandé de représenter graphiquement les différences entre les configurations testées. Cette étape s'inscrit dans la continuité de la fonction `visualiser_statistiques_post_traitement()` utilisée précédemment.

- Écrire une fonction **`visualiser_comparaison_configurations(resultats_configurations)`** qui produit :
 - un barplot comparatif de la taille du vocabulaire par configuration,
 - un diagramme groupé montrant la richesse lexicale et le taux d'hapax,
 - un graphique empilé ou un pie chart illustrant la proportion de bruit lexical selon la configuration.

Ces visualisations permettent d'identifier rapidement la configuration la plus équilibrée entre réduction de bruit, stabilité lexicale et diversité du vocabulaire, avant d'évaluer leurs effets sur les performances en recherche documentaire.